

# SOFTWARE ARCHITECTURE DOCUMENT

Multi-Generation Target Deployment Architecture  
Modern Environments (Tauri 2.0) & Legacy Terminals (wxWidgets + MinGW)

**Target Platforms:** Windows 10/11 (Tauri), Windows XP/7/8 (wxWidgets), Android, Web  
**Core Architecture Strategy:** Shared-Process Async Core & Decoupled Hardware Engines

**Date:** June 2026

**Version:** 2.0.0

# 1. Architectural Strategy & Target Environments

This document details the software architecture design for an enterprise-level Pharmacy Management, Billing, and GST Compliance system. To manage the fragmented hardware common across pharmacies—ranging from modern Windows 11 tablets to legacy, unpatched Windows XP/7 checkout terminals—the system implements a dual-client target framework running against a unified backend schema strategy.

## Dual-Target Execution Matrix

- **Modern Baseline (Windows 10/11, Web, Android):** Deployed via a modern, lightweight web rendering architecture using **Tauri 2.0** with a high-performance **Rust background core**.
- **Legacy Baseline (Windows XP, 7, 8):** Deployed via an ultra-low-memory, standalone **C++ application built on wxWidgets**, cross-compiled cleanly using the **MinGW-w64 GCC toolchain**.

# 2. Functional Requirements & Technical Mappings

| ID     | Module Requirement   | Modern Target Implementation (Tauri)         | Legacy Target Implementation (wxWidgets)             |
|--------|----------------------|----------------------------------------------|------------------------------------------------------|
| REQ-01 | Inventory Management | Rust Core + rusqlite background loop.        | Statically linked native C++ SQLite engine.          |
| REQ-02 | High-Velocity POS    | Web UI + Asynchronous Rust IPC bridge.       | Native Win32 text components + wxGrid ledger.        |
| REQ-03 | GST GSTR-1 Output    | Rust serde_json fast-stream processing.      | C++ nlohmann/json or string buffers.                 |
| REQ-04 | Proactive Alerts     | Tokio async tasks querying DB cycles.        | Isolated wxThread parsing indexed database queries.  |
| REQ-05 | Hardware Access      | HTML5 WebView + Rust system commands.        | Direct Win32 serial port communication (`COM1-4`).   |
| REQ-06 | Thermal Printing     | Rust native spooler system bindings.         | Direct GDI canvas rendering to system printer queue. |
| REQ-07 | GSTR-2B Matching     | Dedicated multi-threaded Rust worker thread. | Independent background wxThread task execution.      |

### 3. Legacy System Decomposition (wxWidgets Client)

To ensure high stability on legacy hardware with limited memory (e.g., 512MB RAM single-core systems), the legacy build discards all web-browser runtime engines. It compiles directly into standard machine-optimized code that communicates using the host operating system's native Win32 controls.

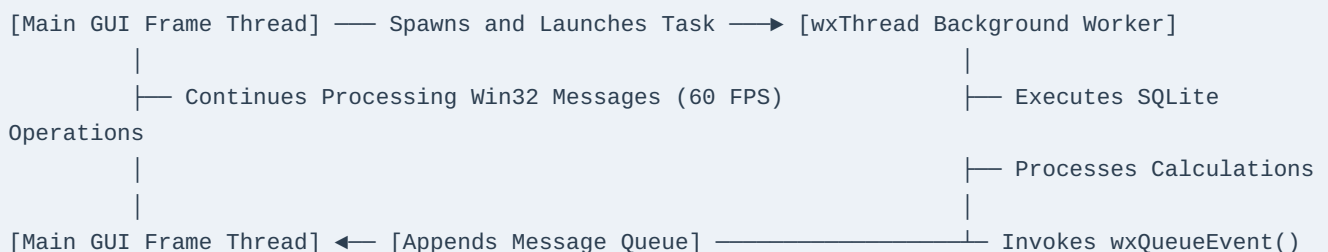
#### 3.1 Compilation & Target Configuration Pipeline

The application is built using the MinGW-w64 environment toolchain. It utilizes complete static linking flags to compile all C++ abstractions, runtime modules, and SQLite dependency trees directly into a single executable file, eliminating any external installation requirements.

```
# MinGW Linker Instruction Flags for Native Legacy Target Compilation
g++ -O3 -std=c++11 main.cpp -static -static-libgcc -static-libstdc++ -Wl,--subsystem,windows:
5.01 -o PharmacyPOS.exe
```

#### 3.2 UI Thread Isolation Model

To keep the billing interface running fluidly at 60 frames per second without stuttering during background tasks, the application uses an event-driven thread architecture:



### 4. Offline-First Synchronization Architecture

Both the modern Tauri-based application and the legacy wxWidgets application share a uniform architectural data schema strategy.

- **Atomic Log Commit Block:** Every change to inventory or ticket items executes inside an isolated database transaction. This step writes the record change to its target table and appends a synchronization payload to the local Sync\_Outbox queue in one atomic action.
- **Data Conflict Strategy:** The system implements a Last-Writer-Wins (LWW) conflict strategy. The database records logical microsecond timestamps down to individual columns, allowing the cloud database layer to automatically merge conflicting updates without locking the application UI.

## 5. Security & Enterprise Isolation Architecture

- **Cloud Multi-Tenant Verification:** Centralized databases (PostgreSQL) use strict Row-Level Security (RLS) layers to filter data streams. This ensures different pharmacies can never see each other's data across sync loops:

```
WHERE tenant_id = current_setting('request.jwt.claim.tenant_id')
```

- **Legacy Access Controls:** On older Windows XP/7 terminals that lack modern browser-based OAuth handling, the application authenticates via secure, signed cryptographic payload handshakes handled by a lightweight local security layer. This validation step occurs over secure TLS 1.2 network connections before transmission loops are approved.